Academic Year: 2026-27 SAP ID: 60003240242

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 23/07/2026**

COURSE NAME: Computer Networks **CLASS: T. Y. B.Tech**

Experiment No. 2

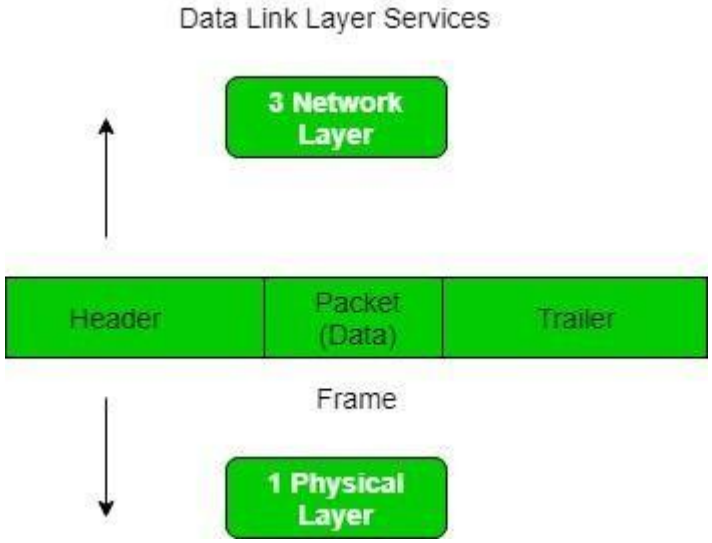
CO/LO: Analyse various concerns at different layers in reference models using suitable techniques.

Aim: Write a program to implement Framing Techniques: Character count, Byte stuffing, Bit stuffing.

Theory:

Frames are the units of digital transmission, particularly in computer networks and telecommunications. Frames are comparable to the packets of energy called photons in the case of light energy. Frame is continuously used in Time Division Multiplexing process.

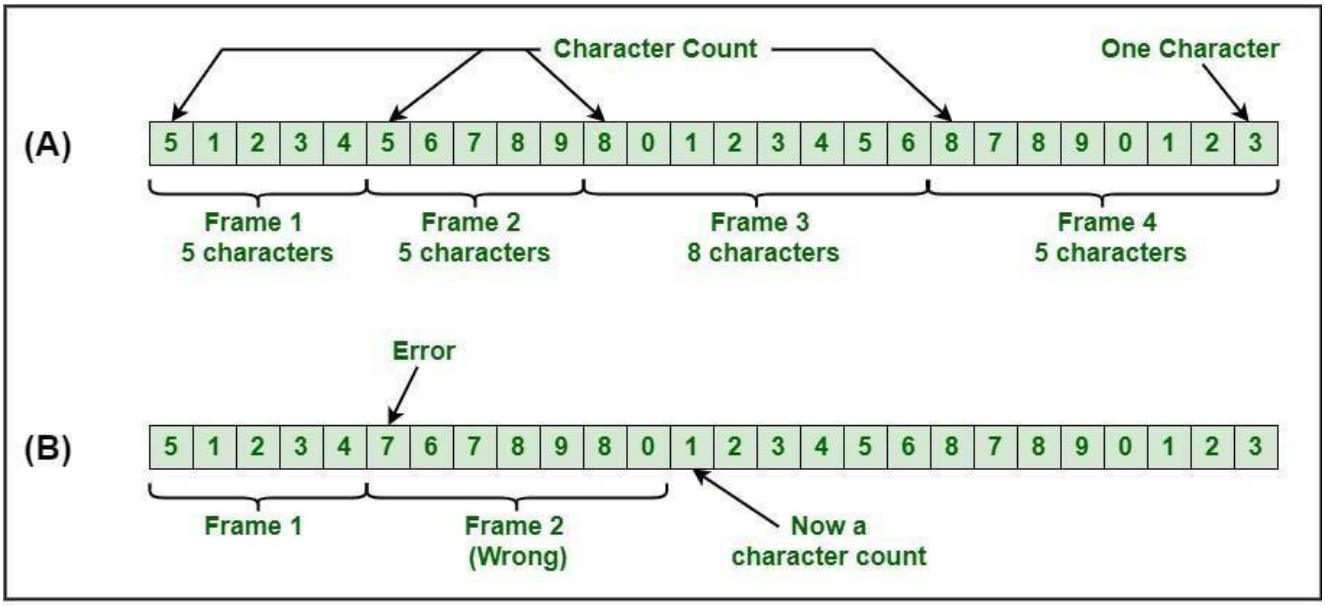
Framing is a point-to-point connection between two computers or devices consisting of a wire in which data is transmitted as a stream of bits. However, these bits must be framed into discernible blocks of information. Framing is a function of the data link layer. It provides a way for a sender to transmit a set of bits that are meaningful to the receiver. Ethernet, token ring, frame relay, and other data link layer technologies have their own frame structures. Frames have headers that contain information such as error-checking codes.



At the data link layer, it extracts the message from the sender and provides it to the receiver by providing the sender's and receiver's addresses. The advantage of using frames is that data is broken up into recoverable chunks that can easily be checked for corruption.

The process of dividing the data into frames and reassembling it is transparent to the user and is handled by the data link layer.
Framing is an important aspect of data link layer protocol design because it allows the transmission of data to be organized and controlled, ensuring that the data is delivered accurately and efficiently.

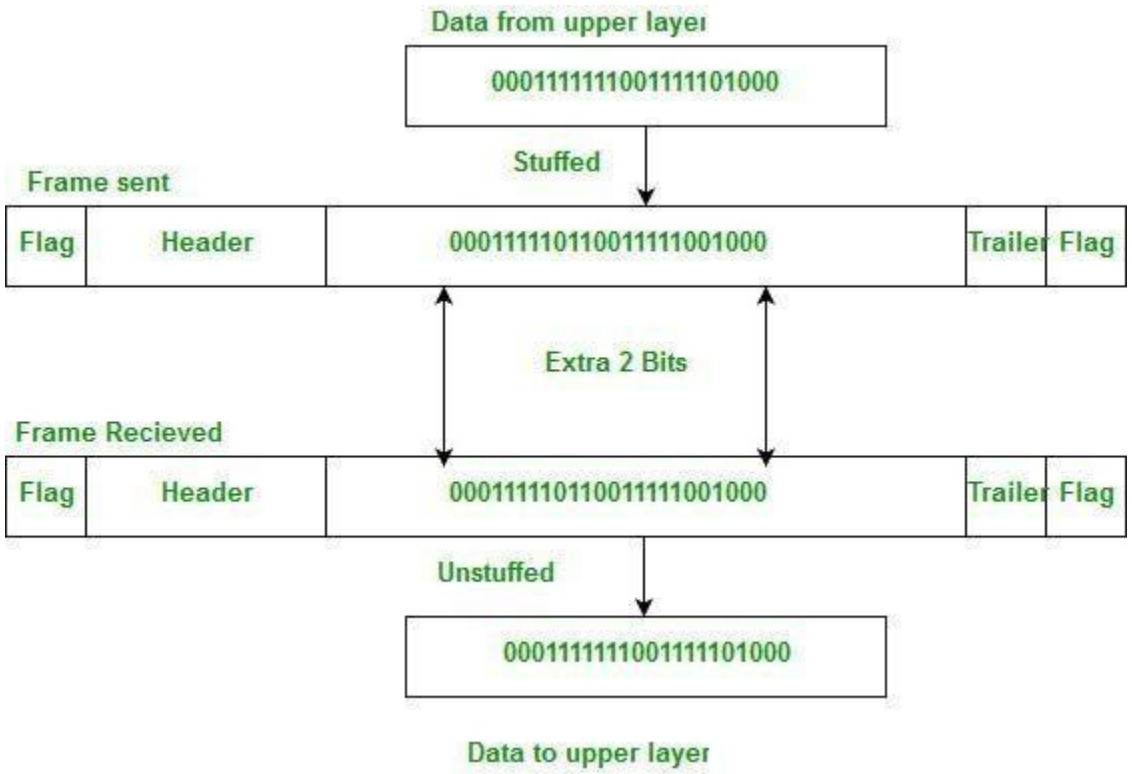
Character Count:
This method is rarely used and is generally required to count total number of characters that are present in frame. This is be done by using field in header. Character count method ensures data link layer at the receiver or destination about total number of characters that follow, and about where the frame ends. There is disadvantage also of using this method i.e., if anyhow character count is disturbed or distorted by an error occurring during transmission, then destination or receiver might lose synchronization. The destination or receiver might also be not able to locate or identify beginning of next frame.



A Character Stream

(A) Without Errors
(B) With one Error

Bit Stuffing:
Bit stuffing is also known as bit-oriented framing or bit-oriented approach. In bit stuffing, extra bits are being added by network protocol designers to data streams. It is generally insertion or addition of extra bits into transmission unit or message to be transmitted as simple way to provide and give signaling information and data to receiver and to avoid or ignore appearance of unintended or unnecessary control sequences.
It is type of protocol management simply performed to break up bit pattern that results in transmission to go out of synchronization. Bit stuffing is very essential part of transmission process in network and communication protocol. It is also required in USB.



Byte Stuffing:

- In this method, start and end of frame are recognized with the help of flag bytes. Each frames starts with and ends with a flag byte. Two consecutive flag bytes indicate the end of one frame and start of the next one. The flag bytes used in the figure 2 used is named as "ESC" flag byte.
- A frame delimited by flag bytes. This framing method is only applicable in 8bit character codes which are a major disadvantage of this method as not all character codes use 8-bit characters e.g. Unicode.
- Four example of byte sequences before and after stuffing:

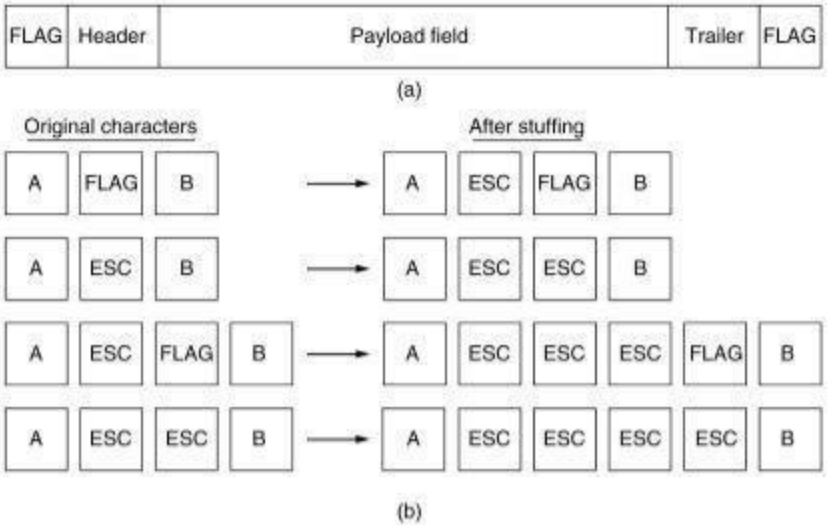


Fig2: Framing with Byte stuffing

Academic Year: 2026-27 SAP ID: 60003240242

Exercises:

Task 1 –
Your program will read a file (in pdf). Convert given files into ASCII and then Binary values whichever applicable. Use this prepared file and create frames of manageable units called frames with following three options –

1. Character count: Randomly generate character count between 2^0 to 2^16. Display the various frames created with the help of preprocessed file as above.

2. Bit Stuffing: Generate Fixed frames of equal length as per used defined input probably Power of 2(2^n) from above preprocessed file by using bit stuffing method.
   Assume (01111110) as Start and End of Frame. Bit "Zero" should be stuffed in payload if 5 consecutive "1" occurs in data. Pad your data sequence (1000000…..0) to make the preprocessed file multiple of 128.

3. Byte Stuffing: Implement variable length byte stuffing using character stuffing method. Insert special character ("ESC") character in preprocessed file at random interval between 2^0 to 2^16. Display the various frames created with the help of preprocessed file as above.

Sr. No	Frame Size	No. of Frames	Overhead (%)
1	64		
2	128		
3	256		

Code:

```
import random
import math
from PyPDF2 import PdfReader

FLAG = "01111110"
ESC = "ESC"
 def
read_pdf(filename):
    reader =
PdfReader(filename)     text =
""     for page in
reader.pages:         if
page.extract_text():
            text += page.extract_text()
return text
 def
text_to_ascii(text):
    return [ord(c) for c in text]
 def
ascii_to_binary(ascii):
    return "".join(format(x, "08b") for x in ascii)
```

```
 def
character_count_framing(text):
    print("\nCharacter Count Framing (First 3 Frames):")
frames = []     i = 0     while i < len(text):
        count = random.randint(1, min(65536, len(text)-i))
frames.append((count, text[i:i+count]))         i += count
print("Total Frames:", len(frames))     for i, (count,
frame) in enumerate(frames[:3], 1):
        print(f"\nFrame {i}")         print("Count:", count)
print("Data:", frame[:50], "..." if len(frame) > 50 else "")
 def
bit_stuffing(bits):
    stuffed = ""     ones = 0
for bit in bits:
stuffed += bit         if bit
== '1':             ones += 1
if ones == 5:
stuffed += '0'
ones = 0         else:
            ones = 0
return stuffed
 def bit_stuffing_frames(binary,
size):
    print("\nBit Stuffing (First 3 Frames):")
stuffed = bit_stuffing(binary)     while
len(stuffed) % 128 != 0:
        stuffed += "0"
    frames = [stuffed[i:i+size] for i in range(0, len(stuffed), size)]
print("Total Frames:", len(frames))     for i, frame in
enumerate(frames[:3], 1):
        print(f"\nFrame {i}")
print(FLAG + frame + FLAG)
 def
byte_stuffing(text):
    print("\nByte Stuffing (First 3 Frames):")     data
= list(text)     interval = random.randint(1,
min(65536, len(data)))     pos = interval     while pos
< len(data):         data.insert(pos, ESC)         pos
+= interval + 1
```

```
    stuffed = []
for ch in data:
if ch == ESC:
            stuffed.extend([ESC, ESC])
elif ch == "F":
            stuffed.extend([ESC, "FLAG"])
else:
            stuffed.append(ch)     frame_size = 30     frames =
[stuffed[i:i+frame_size] for i in range(0, len(stuffed), frame_size)]
print("Total Frames:", len(frames))     for i, frame in enumerate(frames[:3], 1):
print(f"\nFrame {i}")         print("FLAG", " ".join(frame), "FLAG")
 def observation(binary_len):
print("\nObservation:")
print("{:<10}{:<15}{:<15}{:<15}".format(
        "Frame", "No Frames", "Overhead", "Overhead
%"))     for size in [64, 128, 256]:         frames =
math.ceil(binary_len / size)         overhead = frames
* 16         percent = overhead / binary_len * 100
print("{:<10}{:<15}{:<15}{:.2f}".format(
size, frames, overhead, percent))
 def
main():
    filename = input("Enter PDF filename:
")     text = read_pdf(filename)
print("\nPDF Loaded Successfully")
print("Characters:", len(text))
ascii_values = text_to_ascii(text)
print("\nFirst 10 ASCII Values:")
print(ascii_values[:10])
    binary = ascii_to_binary(ascii_values)
print("\nFirst 64 Binary Bits:")     print(binary[:64])
    character_count_framing(text)     size =
int(input("\nEnter Frame Size (64/128/256): "))
bit_stuffing_frames(binary, size)     byte_stuffing(text)
observation(len(binary))
 if __name__ ==
"__main__":
    main()
```

Output:

```
Enter PDF filename: sample_pdf_file.pdf

PDF Loaded Successfully
Characters: 9203

First 10 ASCII Values:
[9, 49, 9, 83, 97, 109, 112, 108, 101, 32]

First 64 Binary Bits:
0000100100110001000010010101001101100001011011010111000001101100

Character Count Framing (First 3 Frames):
Total Frames: 6

Frame 1
Count: 7715
Data:   1      Sample PDF  Created for testing PDFObject  This ...

Frame 2
Count: 1322
Data: Duis sagittis ipsum. Praesent mauris.   Fusce nec  ...

Frame 3
Count: 121
Data: Curae; Morbi lacinia molestie dui. Praesent blandi ...

Enter Frame Size (64/128/256): 128

Bit Stuffing (First 3 Frames):
Total Frames: 576

Frame 1
0111111000001001001100010000100101010011011000010110110101110000011011000110010100000010100000100010001100010000000100000010000011011111110

Frame 2
0111111001110010011001010110000101110100011001010110010000100000011001100111011110110010001000000111010001100101011100110111010001101001011111110

Frame 3
0111111001101110011001110010000001010000010001000100011001001111011000100110101001100101011000110111010000100000001000000101010001101000011111110

Byte Stuffing (First 3 Frames):
Total Frames: 308

Frame 1
FLAG    1     S a m p l e   P D ESC FLAG    C r e a t e d   f o r   t e FLAG

Frame 2
FLAG s t i n g   P D ESC FLAG O b j e c t    T h i s   P D ESC FLAG   i s FLAG

Frame 3
FLAG  t h r e e   p a g e s   l o n g .  T h r e e   l o n g   FLAG

Observation:
Frame     No Frames     Overhead      Overhead %
64        1151          18416         25.01
128       576           9216          12.51
256       288           4608          6.26
```

Task 2 –

A sender wants to transmit the following data stream to a receiver – A B ESC FLAG C D FLAG E ESC F.

- Character Stuffing: Special characters: ESC, FLAG o Frame delimiters: Start and End of frame are marked by FLAG o If ESC or FLAG appears in the data, prefix it with ESC.
- Bit Stuffing: Frame starts and ends with the flag sequence 01111110 o If five consecutive 1s appear in the data, insert a 0 after them.
- Byte Stuffing: Frame delimiters: Start and End marked by byte 0x7E o Escape byte: 0x7D o If data contains 0x7E or 0x7D, prefix it with 0x7D and XOR the byte with 0x20.

1. Apply character stuffing to the given data and show the final frame.
2. Convert the data to binary and apply bit stuffing, showing the final bit stream.
3. Assume ASCII encoding and apply byte stuffing, showing the final byte stream in hexadecimal.

Code:

```
data = ["A", "B", "ESC", "FLAG", "C", "D", "FLAG", "E", "ESC", "F"]
 def
character_stuffing(data):
    stuffed = ["FLAG"]     for ch in
data:         if ch == "ESC" or ch ==
"FLAG":
            stuffed.append("ESC")
stuffed.append(ch)
stuffed.append("FLAG")     return
stuffed
 def
bit_stuffing(data):
    byte_data = []
for ch in data:
if ch == "ESC":
            byte_data.append(0x1B)
elif ch == "FLAG":
            byte_data.append(0x7E)
else:
            byte_data.append(ord(ch))
     binary_data = ''.join(format(b, '08b') for b in
byte_data)     stuffed = ""     count = 0     for bit in
binary_data:
        stuffed += bit
if bit == '1':
count += 1             if
count == 5:
stuffed += '0'
count = 0         else:
            count = 0     flag = "01111110"
final_frame = flag + stuffed + flag
return binary_data, stuffed, final_frame
 def
byte_stuffing(data):
    stuffed = [0x7E]
for ch in data:
if ch == "ESC":
```

```
            byte = 0x1B
elif ch == "FLAG":
byte = 0x7E         else:
            byte = ord(ch)         if
byte == 0x7E or byte == 0x7D:
stuffed.append(0x7D)
stuffed.append(byte ^ 0x20)
else:

stuffed.append(byte)
stuffed.append(0x7E)     return
stuffed
 char_frame = character_stuffing(data)
print("Character Stuffing:") print("
".join(char_frame)) binary, stuffed_bits, final_bits
= bit_stuffing(data) print("\nBinary Data:")
print(binary) print("\nBit Stuffed Data:")
print(stuffed_bits) print("\nFinal Bit Frame:")
print(final_bits) byte_frame = byte_stuffing(data)
print("\nByte Stuffing (Hex):") print('
'.join(f'{b:02X}' for b in byte_frame))
```

Output:

```
PS C:\Users\djsce.student\Desktop\I098> python -u "c:\Users\djsce.student\Desktop\I098\Experiment1_2.py"
Character Stuffing:
FLAG A B ESC ESC ESC FLAG C D ESC FLAG E ESC ESC F FLAG

Binary Data:
0100000101000010000110110111111100100001101000100011111100100010100011011010000110

Bit Stuffed Data:
01000001010000100001101101111101001000011010001000111110100100010100011011010000110

Final Bit Frame:
011111100100000101000010000110110111111010010000110100010001111101001000101000110110100011001111110

Byte Stuffing (Hex):
7E 41 42 1B 7D 5E 43 44 7D 5E 45 1B 46 7E
```

Conclusion: Study of network commands helps to understand the network configuration better.

Reference: www.geeksforgeeks.org